

SQL for Data Analysis

Lecture 1

Ilya Iyapin

BI Developer at Avito

About me

- Bachelor's degree in Economics in HSE
- Master's degree in Management (“Data Analysis for Business and Economics”)
- Worked as a Data Analyst in Kuper (ex SberMarket)
- Currently working as a BI Developer at Avito Auto



How we communicate

1. **Telegram channel**

A preferred way of communication since you can share your problems with others and together we are able to find the best solution

2. **Direct messages in Telegram**

If you don't want to share your problems with others you may ask me directly

3. **Mail: ivlyapin@hse.ru**

I don't really recommend it since I am a rare guest on that domain



Link to a telegram chat

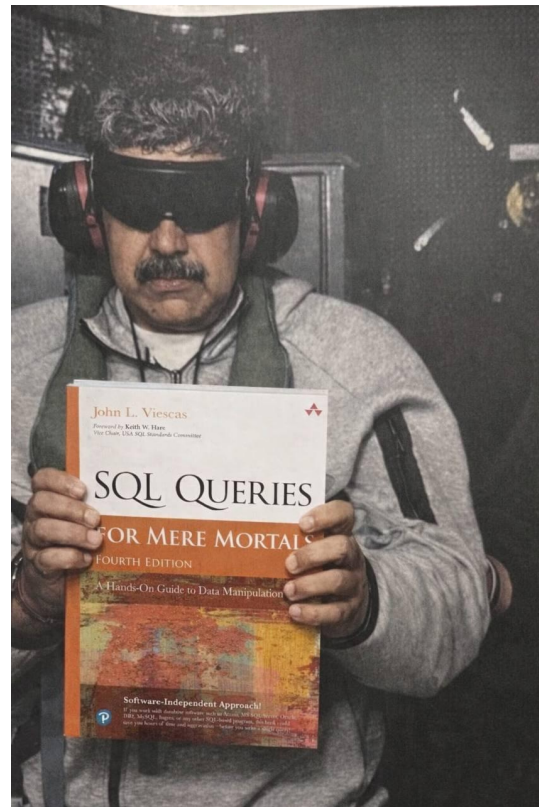
How the grading system works

Final mark = $0.2 * \text{Project 1} + 0.4 * \text{Project 2} + 0.4 * \text{Exam}$

Literature

Generally, the best and most qualitative knowledge that you can obtain is through practice

But if you wish to use some book as a reference to make sure you get everything correctly, I recommend [this](#)



Plan for today

1

Databases. Relational and non-relational databases

2

Introduction to set theory and relational algebra

3

Entity relationship diagrams

4

Q&A

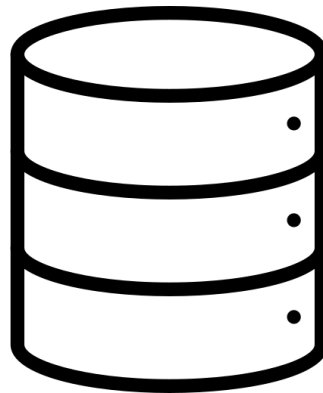
Databases

What types of databases exist and what their properties are

Databases

Database - an organized collection of data that can be easily accessed, managed, and updated

But what does it mean, you may ask?



Databases

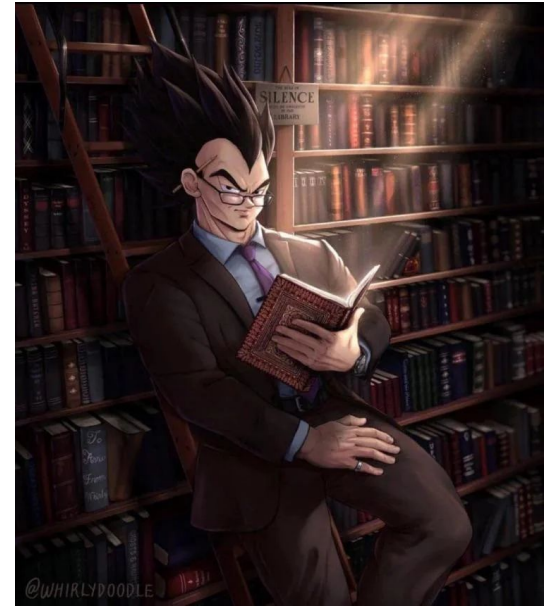
Basically, database is a collection of data with some certain rules.
The rules are strictly defined and depend on what type of data you have



Databases

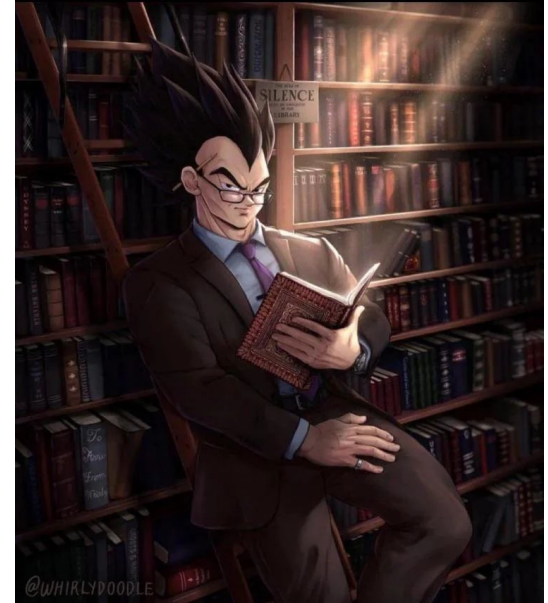
Let's consider the following case, you are a librarian and have a registry of books

What properties do these books have and are they always the same?



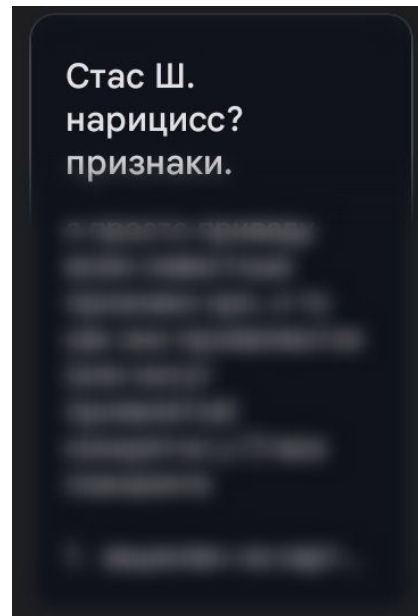
Databases

The answer: usually books have a predefined list of properties - author, ISBN, genre, number of pages and so on
So it is possible to structure them in the same way



Databases

Now let's consider that you as a librarian have an app for taking notes
What do they look like? Do they have properties and are they always the same?

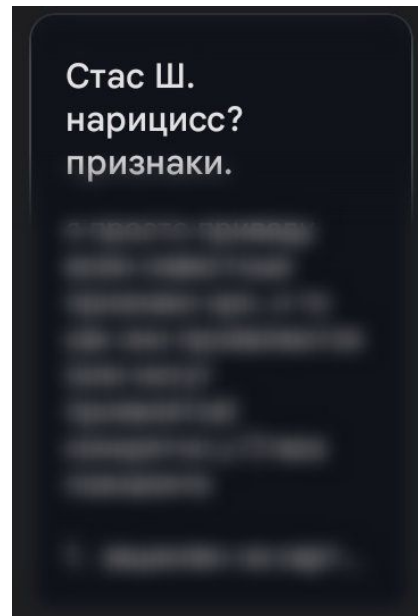


01.01.24 — деньги на ветер

05.01.24 — шутка про картошку

Databases

Answer: usually your notes don't have the same properties and can be anything
However, that is still data that needs to be stored somehow



01.01.24 — деньги на ветер

05.01.24 — шутка про картошку

Databases

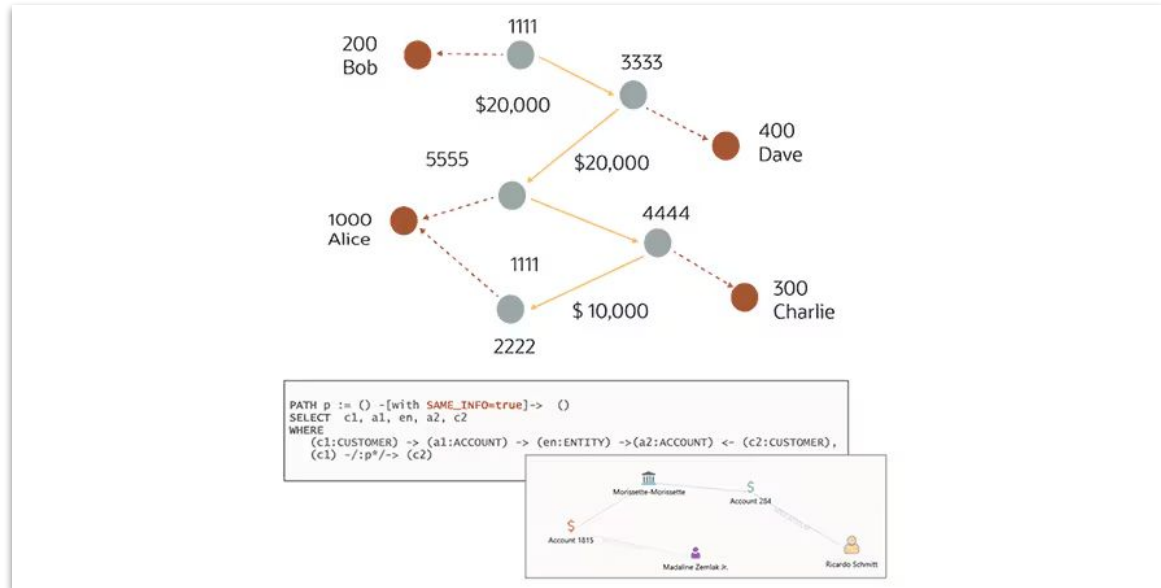
Types of databases

Relational (SQL)	Non-relational (NoSQL)
• Data is organised into structured tables with rows and columns • Fixed schema (precise description of the table's structure, columns and constraints) • Strongly defined relationship between data entities • ACID compliance • SQL support	• Flexible schema • Horizontal scalability (we can just store data on separate servers) • Diverse data models • High performance
Why use? Ensures data integrity and consistency and is suitable for 99% of real world applications	Why use? Able to operate with high amounts of unstructured and changing data, usually used by Big Tech companies
Examples MySQL, PostgreSQL, MariaDB, Microsoft SQL Server, and Oracle Database	Examples MongoDB, Redis, ClickHouse, Vertica

Databases

Non-relational DBs can store data in different ways:

1) As a graph



Source: "What Is a Graph Database?" Oracle Indonesia

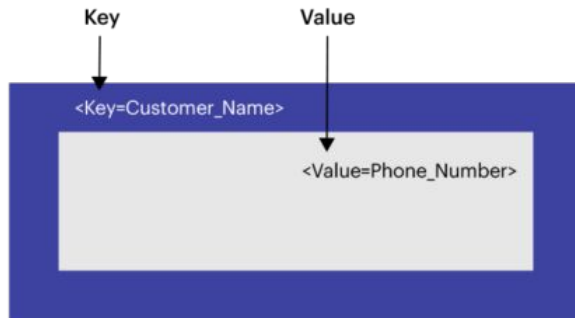
Databases

Non relational DBs can store data in different ways:

2) As a hash table

Phone directory

Key	Value
Paul	(091) 9786453778
Greg	(091) 9686154559
Marco	(091) 9868564334

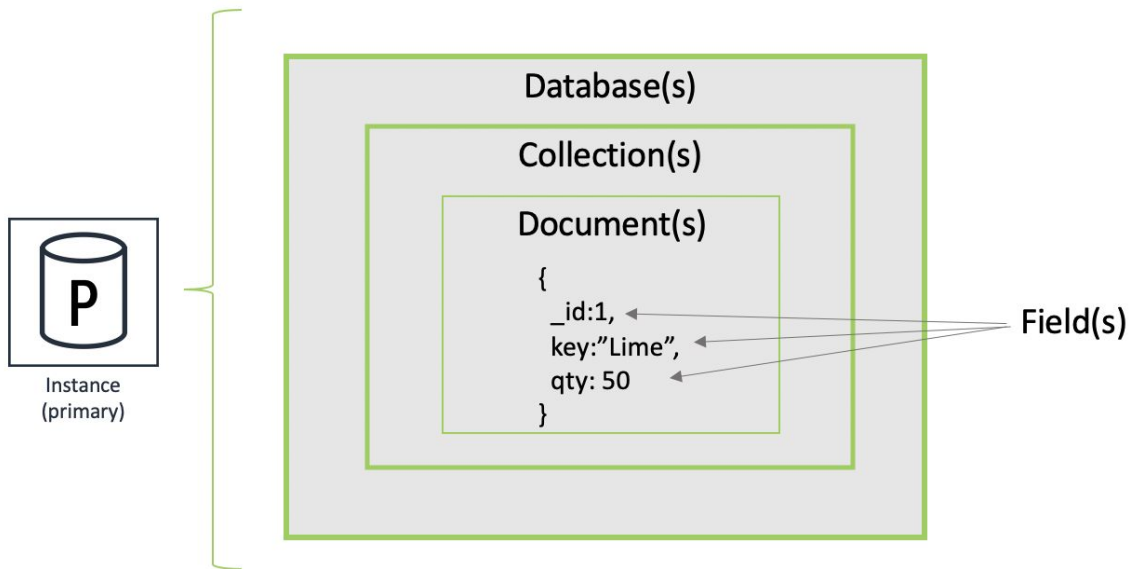


Source: "What Is a Key-Value Database?" Redis

Databases

Non relational DBs can store data in different ways:

3) As documents



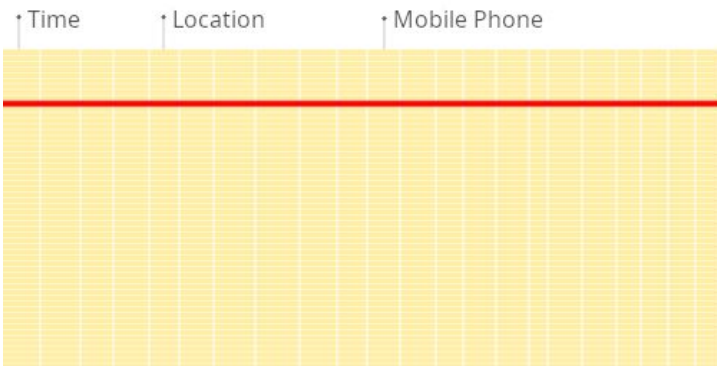
Source: "Document Database", MongoDB

Databases

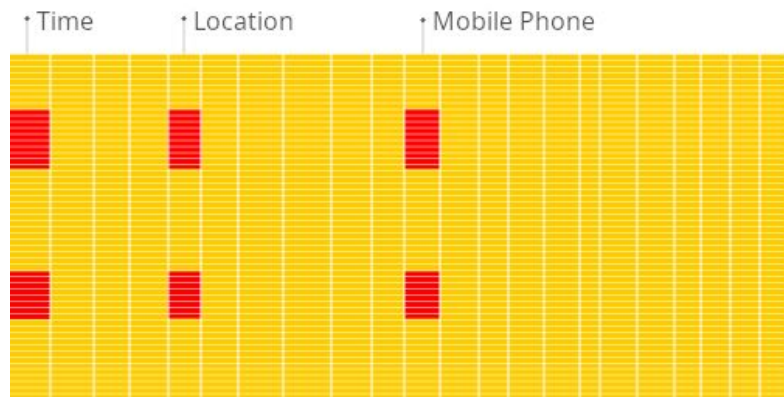
Non relational DBs can store data in different ways:

4) In some sources columnar databases are marked as NoSQL DBs

Traditional row-oriented DB



Column-oriented DB (ClickHouse, for instance)



Source: "What is a columnar database?", ClickHouse

Databases

Relational databases, on the other hand, always look the same and provide ACID

- **Atomicity.** Either all operations of the transaction are reflected properly in the database, or none are.

Example:

You make a bank transaction to your mother and some mistake occurs along the way (your mother's card is blocked). Two requests are made - money withdrawal and deposition to your mother's account. The second one was aborted because of the blocked card. This is not good, since both operations should be aborted. That is why atomicity principle should hold

Databases

- **Consistency.** Execution of a transaction in isolation preserves the consistency of the database

Example:

The consistency requirement here is that the sum of *money transferred* and *money received* be unchanged by the execution of the transaction. Without the consistency requirement, money could be created or destroyed by the transaction! It can be verified easily that, if the database is consistent before an execution of the transaction, the database remains consistent after the execution of the transaction.

Databases

- **Isolation.**

The isolation property of a transaction ensures that the concurrent execution of transactions results in a system state that is equivalent to a state that could have been obtained had these transactions executed one at a time in some order

Example:

Imagine you have a transaction where you take money from the bank account and then roll back your transaction. All of this happens in one transaction. Another person also does some operation with your account and if this transaction occurs at the same time with your transaction then the person will get the wrong data. That's why isolation matters

Databases

- **Durability.**

After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

Example:

Imagine you got your salary, the transaction of transferring money completed successfully and then after a system failure the bank just lost your money

Introduction to set theory and relational algebra

Why they matter and how are related to relational databases

By the way, don't overthink it since this topic is strictly here to demonstrate that many operations from set theory apply to relational databases

Set theory

Why it matters?

- Provides the logical structure for organizing data into tables.
- Helps define relationships between data entities.
- Forms the basis for relational algebra operations.

Set theory

Concept	Meaning	Example
Set	A collection of distinct elements	$\{1,2,3\}$ or $\{\text{'Tyler the Creator'}, \text{'Knocked Loose'}, \text{'Doechii'}\}$
Subset	A set where all elements are part of another set	$\{1,2\}$ is a subset of $\{1,2,3\}$
Union	Combines all unique elements from two sets	$\{1, 2\} \cup \{2, 3\} = \{1, 2, 3\}$
Intersection	Finds common elements between two sets	$\{1, 2\} \cap \{2, 3\} = \{2\}$
Difference	Finds elements in one set that are not in another	$\{1, 2\} \setminus \{2, 3\} = \{1\}$
Cartesian Product	Produces all ordered pairs of elements from two sets	$\{1, 2\} \times \{\text{'A'}, \text{'B'}\} = \{(1, \text{'A'}), (1, \text{'B'}), (2, \text{'A'}), (2, \text{'B'})\}$

Relational algebra

Relational Algebra is a procedural query language. Relational algebra mainly provides a theoretical foundation for relational databases and SQL. The main purpose of using Relational Algebra is to define operators that transform one or more input relations into an output relation

Basically, it is just a query language that uses algebraic expressions to model data

Relational algebra

It consists of the following main operators:

- **Selection (σ):** Filters rows based on a condition
Example: $\sigma(\text{age} > 23)(\text{Employees})$
- **Projection (π):** Selects specific rows from the table
Example: $\pi(\text{name, salary, age})(\text{Employees})$ retrieves the name, age columns.
- **Union ($\cup$):** Combines rows from two relations with the same structure
Example: $R1 \cup R2$ combines all rows from $R1$ and $R2$.
- **Set Difference ($-$):** Finds rows in one relation that are not in another
Example: $R1 - R2$.
- **Join ($\bowtie$):** Combines rows from two relations based on a condition
Example: $\text{Employees} \bowtie \text{Department}$ links employees to their departments.
- **Rename (ρ):** Renames a relation or its attributes
Example: $\rho(A, B)(R)$ will rename attribute B to A of relation R

Relational algebra

Selection example

A	B	C
1	2	4
2	2	3
3	2	3
4	3	4

→ $\sigma(c > 3)(R)$ →

A	B	C
1	2	4
4	3	4

Relational algebra

Projection example

A	B	C
1	2	4
2	2	3
3	2	3
4	3	4

→ $\pi(B,C)(R)$ →

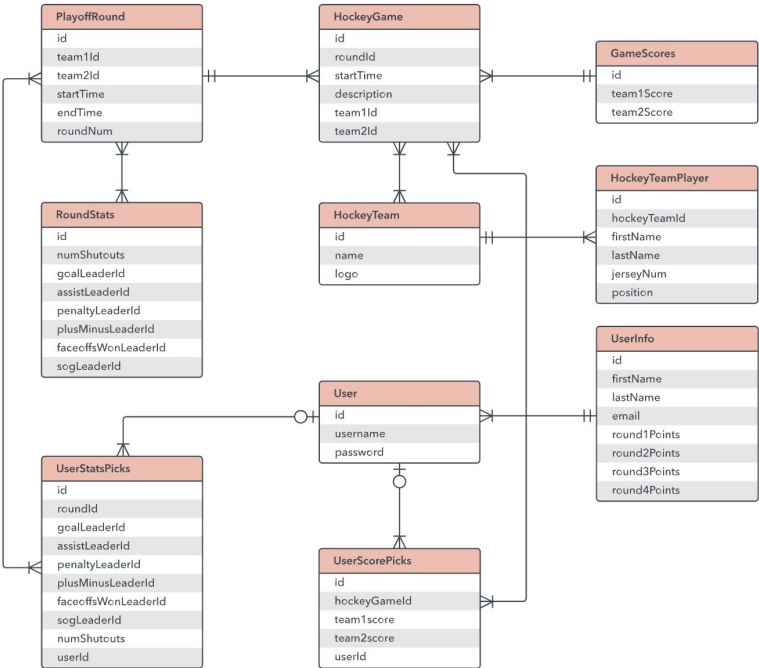
B	C
2	4
2	3
3	4

Entity-relationship diagrams

What they denote and how can be useful for database modeling

ER diagrams

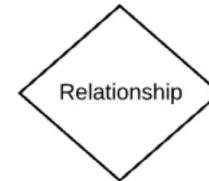
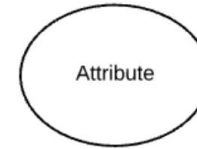
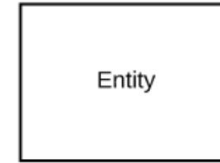
An Entity Relationship (ER) Diagram is a type of flowchart that illustrates how “entities” such as people, objects or concepts relate to each other within a system



ER diagrams

ER diagram is comprised of several components:

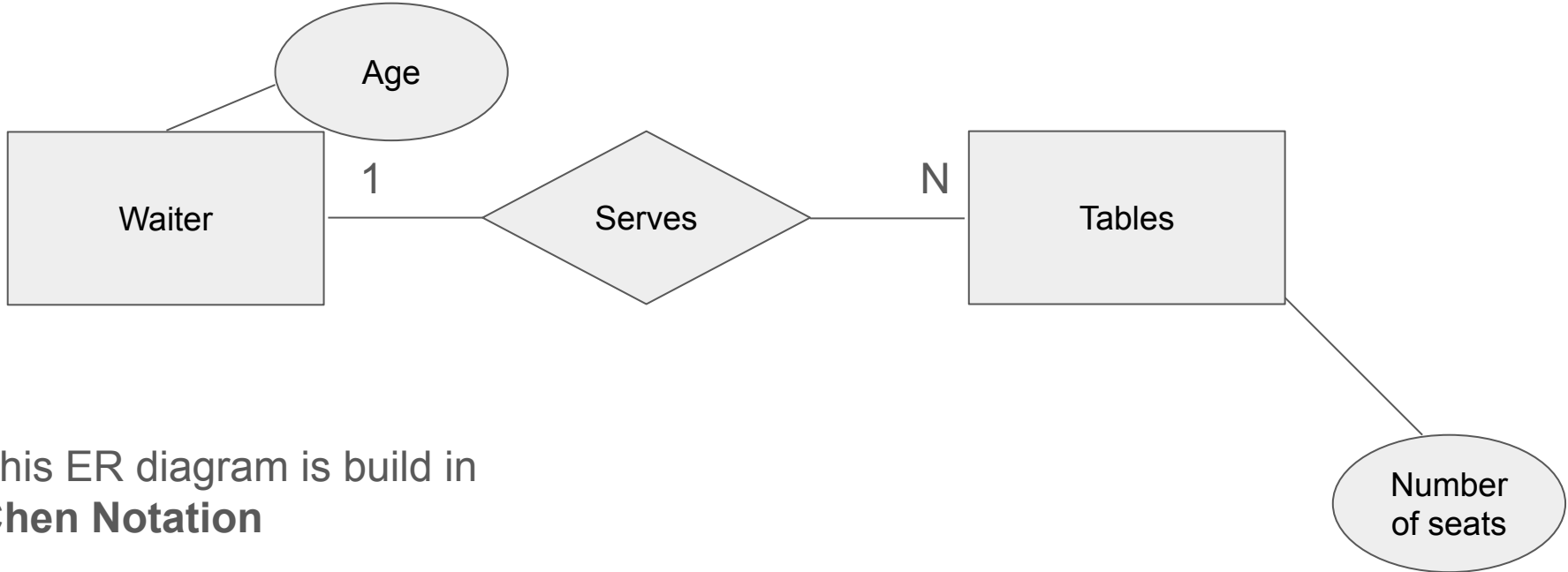
1. **Entity** models real-life objects like students, books, movies and so on. Denoted as a rectangle on an ER diagram
2. **Attributes** describe properties of an entity (e.g. name, title, rating). Denoted as ellipses
3. **Relationships** describe how entities are related to each other
4. **Cardinality** refers to the maximum number of times an instance in one entity can relate to instances of another entity. Its denotations vary across different types of notations being used. The most common types of cardinality are “one-to-one”, “one-to-many”, “many-to-many”



ER diagrams

So we can create something like this

1 and N refer to one-to-many type of cardinality



This ER diagram is build in
Chen Notation

ER diagrams

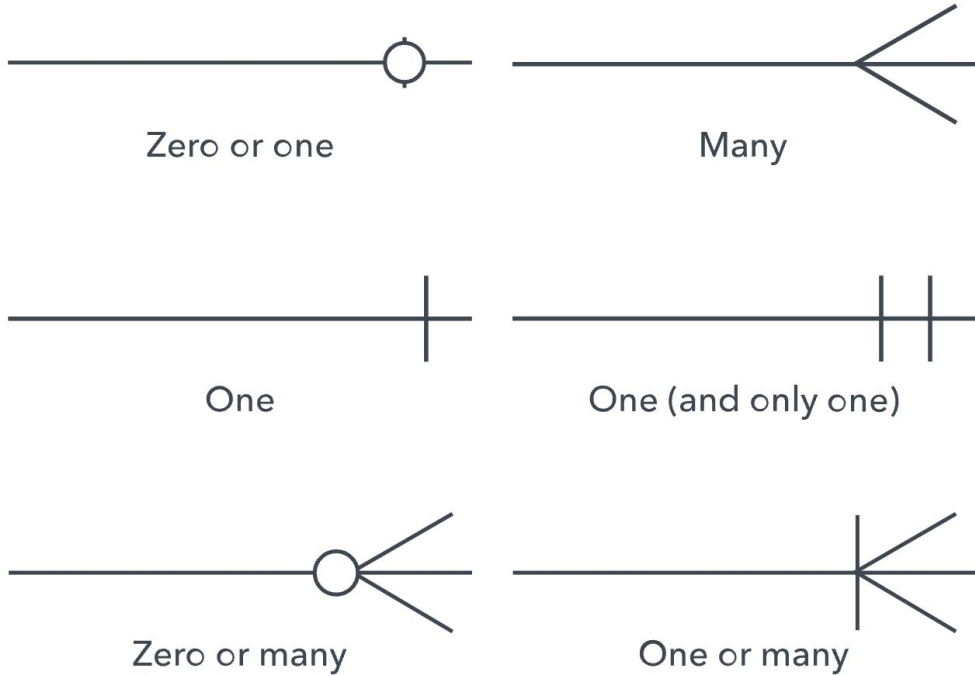
There are roughly 2 types of the most popular notations:

- 1. Chen Notation
- 2. Crow's Foot Notation

	Chen Notation	Crow's Foot Notation
Entity Representation	Rectangle	Rectangle
Attributes	Separate ellipses connected to entities	Listed within the rectangle
Relationships	Diamonds	Lines with symbols
Cardinality Representation	Explicit textual annotations (e.g., "1", "N")	Symbols (lines, circles, crow's feet)
Complexity	Can become cluttered with many attributes	Compact and scalable
Best Use Case	Academic learning and small diagrams	Professional and complex diagrams

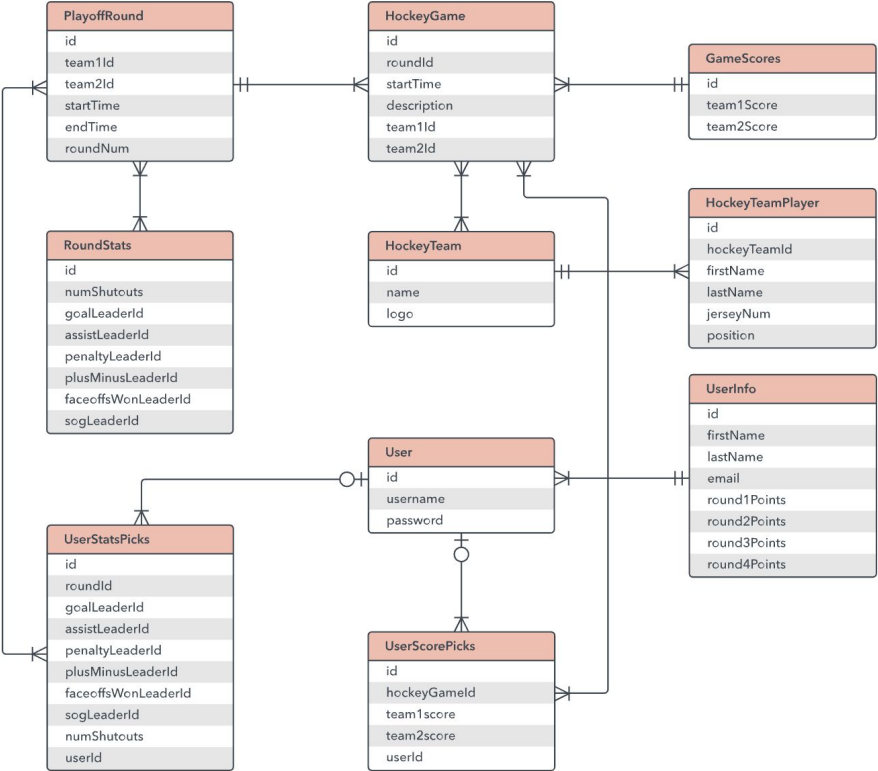
ER diagrams

Cardinality symbols
for Crow's Foot
Notation



ER diagrams

This is what ER diagrams look like in Crow's Foot Notation



Primary and foreign keys

Suppose we have 2 tables that we need to connect with each other

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNT	STUD_AGE
1	RAM	9865278251	Haryana	India	20
2	RAM	9655470231	Punjab	India	19
3	SUJIT	7514290359	Rajasthan	India	18
4	SURESH	8564103258	Punjab	India	21

STUD_NO	COURSE_NO	COURSE_NAME
1	C1	DBMS
2	C2	Computer Networks
1	C2	Computer Networks

Primary and foreign keys

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNT	STUD_AGE
1	RAM	9865278251	Haryana	India	20
2	RAM	9655470231	Punjab	India	19
3	SUJIT	7514290359	Rajasthan	India	18
4	SURESH	8564103258	Punjab	India	21

STUD_NO - is a primary key, which ensures the data in this specific column is **unique** and **non-null**

Primary and foreign keys

STUD_NO	COURSE_NO	COURSE_NAME
1	C1	DBMS
2	C2	Computer Networks
1	C2	Computer Networks

However, in this table `STUD_NO` is a foreign key that serves only as a linker between `STUDENT_COURSE` table and `STUDENT` table. It should not have the properties of the primary key (non-null and unique)

Primary and foreign keys

PRIMARY KEY	FOREIGN KEY
A primary key is used to ensure data in the specific column is unique.	A foreign key is a column or group of columns in a relational database table that provides a link between data in two tables.
It uniquely identifies a record in the relational database table.	It refers to the field in a table which is the primary key of another table.
Only one primary key is allowed in a table.	Whereas more than one foreign key is allowed in a table.
It is a combination of UNIQUE and Not Null constraints.	It can contain duplicate values and a table in a relational database.
It does not allow NULL values .	It can also contain NULL values.

Thank you for attention!

